Signal-Android Architecture

Existing architecture, features, and integration with Signal-Server (REST, WebSocket, CDN, Storage, CDS, SVR, calling).

> Signal-Android is a multi-module Gradle app: feature-oriented MVVM UI, manual service-locator DI (`AppDependencies`), SQLCipher stores, JobManager for durable work, and a dual network stack (libsignal WebSockets + OkHttp REST) against Signal-Server.

MVVM +	**No Hilt**	**SQLCipher**	**WS + REST**
Jobs / service locator	AppDependencies graph	Primary message DB	Chat transport

1. Mental model

UI (Activities / Compose / ViewModels) → Repositories + MessageSender → JobManager (~192 jobs) → libsignal encrypt → WebSocket / REST → Server

Inbound is the reverse: FCM wake and/or persistent WebSocket → `IncomingMessageObserver` → decrypt → `PushProcessMessageJob` → DB + notifications.

2. Gradle modules

Area	Modules	Role
App	`:Signal-Android` (`app/`)	UI, jobs, DB, orchestration
Core	`core:util` , models, network, ui, serialization	Shared primitives
Lib	`lib:libsignal-service` , `lib:network` , video, billing, archive…	Protocol client + media/billing
Feature	`feature:registration` , camera, media-send	Extracted feature libs

3. Client layers

Presentation UI

MainActivity (Compose shell), ConversationActivity, PassphraseRequiredActivity gate, ~248 ViewModels. Feature packages: `conversation/` , `stories/` , `payments/` , `registration/` , `webrtc/` .

Application services DI

`AppDependencies` + `ApplicationDependencyProvider`. `NetworkDependenciesModule` is resettable when config/identity changes.

Durable domain work Jobs

JobManager with FastJobStorage → JobDatabase. Reserved runners for send, process-message, attachment upload, typing, receipts. Chaining: compress → upload → send.

Data SQLCipher

`SignalDatabase` (*Table classes, not Room). Also JobDatabase, KeyValueDatabase/SignalStore. LiveRecipientCache + DatabaseObserver feed UI.

Crypto / protocol libsignal

ACI + PNI stores (sessions, prekeys, Kyber, identity, sender keys). MessageDecryptor / SignalServiceCipher, sealed sender certs, zkgroup for profiles & groups.

Network Dual stack

Legacy: PushServiceSocket + SignalServiceAccountManager / MessageSender / MessageReceiver. Newer: SignalRestClient + MessageApiV2, KeysApiV2, AttachmentApi, CdsApi…. Real-time: LibSignalChatConnection auth + unauth websockets.

Startup (ApplicationContext)

SQLCipher + SignalDatabase → SignalStore → AppDependencies → Migrations → IncomingMessageObserver → RingRTC / Glide / background jobs

4. Feature map

Feature	Primary packages	Server touchpoints
1:1 messaging	`messages/`, `conversation/`, `jobs/*SendJob*`	WS inbox + PUT /v1/messages/{id}
Groups (GV2)	`groups/`, GroupManagerV2	cert/auth/group → Storage Service
Media / stickers	`mediasend/`, `stickers/`, `attachments/`	GET /v4/attachments/form/upload → CDN
Calls 1:1 + group	`service/webrtc/`, RingRTC	signaling msgs + /v2/calling/relays + SFU
Stories	`stories/`, StoryMessageProcessor	messages with ?story=; sealed multi-recipient

Feature	Primary packages	Server touchpoints
Registration	`registration/` , feature:registration	/v1/verification → /v1/registration
Contacts (CDS)	`contacts/sync/` , CdsApi	GET /v2/directory/auth → CDSI
Profiles / usernames	`profiles/` , ProfileApi	/v1/profile, username APIs
Storage sync	`storage/StorageSyncHelper`	GET /v1/storage/auth → Storage Service
PIN / reglock (SVR)	`pin/` , `lock/v2/`	GET /v2/svr/auth → SVR2
Backups / archive	`backup/` , `backup/v2/` , lib:archive	/v1/archives + CDN3
Payments (MobileCoin)	`payments/`	GET /v1/payments/auth
Donations / badges	`badges/` , `subscription/`	donation / subscription controllers
Link / transfer device	`linkdevice/` , lib:device-transfer	WS /v1/websocket/provisioning/

5. Server ecosystem (client view)

Hosted by Signal-Server (chat)

Accounts, devices, messages queue, keys, profiles metadata, attachment upload forms, certificates, CDS/Storage/SVR auth tokens, calling relays, remote config, donations.

External hosts (client talks directly)

CDN 0/2/3 (attachments), Storage Service (groups + secure storage), CDSI enclave, SVR2, SFU/TURN (VoIP), payments service.

6. Transports

> Controllers are mounted on both HTTP Jersey and the authenticated WebSocket Jersey env — the Android client can invoke the same REST paths over HTTP or as framed WebSocket requests while connected.

WebSocket — primary inbox

`/v1/websocket/`

Server pushes `PUT /api/v1/message` envelopes; empty queue → `PUT /api/v1/queue/empty` . Client: LibSignalChatConnection + IncomingMessageObserver.

FCM — wake only

`PUT /v1/accounts/gcm/`

When peer is offline, MessageSender schedules FCM. App: FcmReceiveService → reconnect WS → drain queue. Push is not the message body channel.

REST — OkHttp (no Retrofit)

PushServiceSocket + SignalRestClient. Used for registration, keys, profiles, attachment forms, credentials for external services, and some sends.

7. REST surface Android uses

Domain	Path prefix	Notes
Verification	/v1/verification	Session + SMS/voice/push challenges
Registration	/v1/registration	Create account; optional reglock → 423 + SVR
Accounts / devices	/v1/accounts, /v1/devices, /v2/accounts	GCM, attributes, whoami, link device
Messages	/v1/messages	PUT destination; multi_recipient; stories
Keys	/v2/keys	Upload / fetch prekeys + PQ
Profiles	/v1/profile	Versioned profile + avatar form
Attachments	/v4/attachments/form/upload	Returns CDN# + upload form (2=GCS, 3=TUS)
Certificates	/v1/certificate	Sealed-sender delivery cert; group ZK auth
CDS auth	/v2/directory/auth	Then talk to CDSI host
Storage auth	/v1/storage/auth	Then Storage Service (groups)
SVR2	/v2/svr/auth	PIN / registration lock
Calling	/v2/calling/relays	TURN; call-link create-auth separate
Config	/v2/config	Remote feature flags
Archives	/v1/archives	Message backup auth / media

8. Default hosts (prod BuildConfig)

Service	URL key	Typical host
Chat	SIGNAL_URL	https://chat.signal.org
Storage	STORAGE_URL	https://storage.signal.org
CDN 0/2/3	SIGNAL_CDN*_URL	cdn / cdn2 / cdn3.signal.org
SFU	SIGNAL_SFU_URL	https://sfu.voip.signal.org
CDSI	SIGNAL_CDSI_URL	CDSI enclave host
SVR2	SIGNAL_SVR2_URL	SVR2 host

Staging flavor overrides to `*.staging.signal.org` and sets `LIBSIGNAL_NET_ENV = STAGING`. Assembly point: `SignalServiceNetworkAccess`.

9. Send / receive flows

Outbound message

UI compose → MessageSender → DB → Compress / Upload jobs →
IndividualSendJob / PushGroupSendJob → Encrypt (libsignal) → WS or PUT /v1/messages

Attachments: obtain form from AttachmentControllerV4 → upload to CDN → pointer embedded in envelope. Sealed sender uses UAK or group-send endorsement tokens.

Inbound message

Server queues envelope → Peer online? → WS push → Else FCM wake →
IncomingMessageObserver → MessageDecryptor → PushProcessMessageJob → processors

Processors include DataMessageProcessor, SyncMessageProcessor, StoryMessageProcessor, CallMessageProcessor. UI updates via DatabaseObserver / LiveRecipient.

Server-side delivery (WebSocketConnection)

Connect

Upgrade with Basic auth → AuthenticatedConnectListener loads Account+Device → WebSocketConnection subscribes to MessagesManager stream. Conflict closes with 4409 Connected elsewhere.

Drain

Each envelope: PUT /api/v1/message (protobuf). Client ACK → store ack + optional delivery receipt. On disconnect with pending messages, delayed push notification is scheduled (~1 min).

Registration sketch

/v1/verification session → Code / push challenge → POST /v1/registration →
Store ACI+PNI+password → PUT /v1/accounts/gcm → Upload keys / profile

10. Auth & config

> Not Bearer tokens. Ongoing API auth is HTTP Basic: username `{ACI}` or `{ACI}.{deviceId}`, password = device auth token. Same scheme for REST, WebSocket upgrade, and gRPC.

Mechanism	Where	Purpose
Device Basic auth	SignalStore.account() → DynamicCredentialsProvider	All authenticated chat APIs
Unidentified-Access-Key	SealedSenderAccessUtil	Anonymous sealed-sender send / key fetch
Group send token	GroupSendUtil / endorsements	Multi-recipient without UAK XOR
External short-lived creds	Storage / CDS / SVR / Payments auth endpoints	Talk to sibling services
Registration session	VerificationController	Pre-account phone proof
Registration lock	SVR2 + /accounts/registration_lock	423 on wrong PIN at re-register

Build flavors

- **distribution:** play (default), website, github, nightly
- **environment:** prod (default) · staging (applicationId suffix .staging, staging hosts)

`SignalServiceNetworkAccess.getConfiguration(e164)` may switch to domain-fronting (Fastly / Google reflectors) for censored country codes, static IP DNS fallbacks, and optional user proxy from `SignalStore.proxy`.

11. Key classes

Android — start here

Class	Module / package	Role
ApplicationContext	app	Boot: DB, DI, IncomingMessageObserver
AppDependencies	dependencies/	Service locator (jobs, sockets, stores)
IncomingMessageObserver	messages/	Keep WS alive; decrypt; enqueue process jobs
MessageSender	sms/	Persist outgoing; chain send jobs
SignalServiceMessageSender	libsignal-service	Encrypt + send (sealed, SKDM, multi)
SignalRestClient / MessageService	lib:network	Newer REST + send helpers
SignalServiceNetworkAccess	push/	Build SignalServiceConfiguration URLs
JobManager	jobmanager/	Durable retries / constraints
SignalDatabase	database/	SQLCipher message/recipient/attachment tables
SignalCallManager	service/webrtc/	RingRTC + signaling orchestration

Server — start here

Class	Package	Role
WhisperServerService	textsecuregcm	Dropwizard entry; register controllers + WS
MessageController	controllers	PUT /v1/messages send paths
AuthenticatedConnectListener	websocket	On WS connect → WebSocketConnection
WebSocketConnection	websocket	Push envelopes to client; queue empty
MessageSender (server)	push	Insert queue; FCM if offline
AccountAuthenticator	auth	Basic ACI.deviceId + password
KeysController	controllers	/v2/keys prekey store
AttachmentControllerV4	controllers	CDN upload forms

12. Practical tips for self-hosting

1. Point the `staging` flavor (or override BuildConfig URLs) at your chat/CDN/storage/CDSI/SVR hosts via `SignalServiceNetworkAccess`.
2. Treat WebSocket + JobManager as the backbone — not a Retrofit API list.
3. Account identity is ACI + PNI + device password in SignalStore.
4. Groups and much contact state live on Storage Service / CDS, not only on the chat server.

Sources: Signal-Android (`org.thoughtcrime.securesms`) · Signal-Server (`WhisperServerService`) · documented Jul 2026